

# Diabetes Prediction

---

This notebook presents a complete machine learning pipeline developed for competition on health data. The goal is to predict the probability of a diabetes diagnosis from a set of patient health indicators, using a synthetically generated dataset derived from the Diabetes Health Indicators Dataset.

The work is structured to cover the full ML workflow: exploratory analysis, preprocessing, feature engineering, model training and comparison, evaluation and final prediction.

**Source competition:** <https://www.kaggle.com/competitions/playground-series-s5e12>

**Author:** Sansone Lorenzo — University of Bologna, Academic Year 2025/2026

## Notebook Structure

---

1. Introduction
2. Setup
3. Exploratorion of Data & Cleaning
4. Preprocessing Dataset
5. Models & Parameters Definition
6. Model Training
7. Evaluation & Comparison
8. Conclusions
9. References

# 1. Introduction

---

This notebook addresses a binary classification problem: **predicting the probability that a patient has been diagnosed with diabetes**, based on a set of health indicators.

The dataset was synthetically generated from a deep learning model trained on the [Diabetes Health Indicators Dataset] (<https://www.kaggle.com/datasets/mohankrishnathalla/diabetes-health-indicators-dataset/data>), meaning feature distributions are close — but not identical — to the original.

---

## 1.1 Objective

The goal of this competition is to leverage machine learning techniques to predict the probability that a patient will be diagnosed with diabetes based on various clinical and demographic features.

Predict the **probability** of `diagnosed_diabetes` for each sample in the test set. The primary evaluation metric is **ROC-AUC** (Area Under the Receiver Operating Characteristic Curve).

---

## 1.2 Files

| File                  | Description                                 |
|-----------------------|---------------------------------------------|
| train.csv             | Training set with features and target label |
| test.csv              | Test set — features only, no labels         |
| sample_submission.csv | Expected output format for predictions      |

## 2. Setup

---

### 2.1 Install Missing Dependencies (if needed)

The cell below installs any libraries not available by default in the environment.  
If you're running this on Kaggle or Google Colab, most of these are pre-installed.

```
In [1]: # Install libraries not available by default
        # Uncomment as needed depending on the environment

        # !pip install numpy
        # !pip install -U scikit-learn
        # !pip install pandas
        # !pip install seaborn
        # !pip install matplotlib
```

## 2.2 Import Libraries

In [ ]:

```
# — Core


---


import numpy as np                # Numerical operations
import pandas as pd               # Data manipulation
from sklearn.preprocessing import OrdinalEncoder, OneHotEncoder #
Encoding features
from sklearn.model_selection import train_test_split # Dataset split
from sklearn.model_selection import ParameterGrid
from sklearn.metrics import roc_auc_score
from sklearn.metrics import classification_report
from sklearn.metrics import RocCurveDisplay
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split, GridSearchCV,
StratifiedKFold

# — Models


---


from sklearn.svm import SVC
from sklearn.linear_model import Perceptron
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import AdaBoostClassifier, RandomForestClassifier
import lightgbm as lgb
from xgboost import XGBClassifier
from catboost import CatBoostClassifier

# — Visualization


---


import matplotlib.pyplot as plt   # Base plotting library
import matplotlib.ticker as mticker
import seaborn as sns             # Statistical visualizations

# — Utilities


---


import os
import time
from IPython.display import display
import warnings

warnings.simplefilter(action='ignore', category=FutureWarning)
print("✅ All libraries imported successfully.")
```

✅ All libraries imported successfully.

## 2.3 Global Configuration

Centralizing configuration constants makes experiments reproducible and easy to tweak.

- `RANDOM_STATE` : fixes all stochastic operations so results are reproducible
- `TRAIN_SIZE` : split percentage for training set
- `N_FOLDS` : number of folds for cross-validation
- `TARGET` : name of the column we want to predict

```
In [3]: # — Reproducibility
        _____
RANDOM_STATE = 42          # Fixed seed for all random operations
np.random.seed(RANDOM_STATE)

# — Hold-Out Method
        _____

TRAIN_SIZE = 0.67

# — Cross-Validation
        _____

N_FOLDS = 5                # Number of StratifiedKFold folds

# — Dataset
        _____

TARGET = 'diagnosed_diabetes' # Name of the target column
DATA_DIR = '/kaggle/input/competitions/playground-series-s5e12' #Adjust
according the dataset location

print(f"Configuration set:")
print(f"  Random state : {RANDOM_STATE}")
print(f"  CV folds      : {N_FOLDS}")
print(f"  Target col     : '{TARGET}'")
```

```
Configuration set:
  Random state : 42
  CV folds      : 5
  Target col    : 'diagnosed_diabetes'
```

## 2.4 Load the Data

We load the train CSV file (supervised data) provided by the competition:

- `train.csv` : used to train, validate and test our models

In [4]:

# — Load Datasets

```
df = pd.read_csv(f'{DATA_DIR}/train.csv')

print("✅ Files loaded successfully.")
print(f"\n{'Dataset':<22} {'Rows':>8} {'Columns':>10}")
print("-" * 42)
print(f"{'train.csv':<22} {df.shape[0]:>8} {df.shape[1]:>10}")
```

✅ Files loaded successfully.

| Dataset   | Rows   | Columns |
|-----------|--------|---------|
| train.csv | 700000 | 26      |

### 3. Exploration of Data & Cleaning

Before any statistical analysis, we take a first look at the raw data to verify it matches what the competition describes: correct number of rows and columns, expected feature names, and sensible values at a glance.

#### 3.1 General information about dataset

The dataset includes patient profiles with features based on **demographics, lifestyle habits, family history** and **clinical measurements** that are well-established indicators of diabetes risk.

All data has been synthetically generated using statistical distributions and correlations derived from real-world medical research studies. This approach ensures:

- 🛡️ Privacy preservation (no real patient data used)
- 📊 Realistic health patterns and relationships
- 🛠️ Suitability for machine learning, analysis, and experimentation

Display the list of columns

In [5]: `df.columns`

```
Out[5]: Index(['id', 'age', 'alcohol_consumption_per_week',
              'physical_activity_minutes_per_week', 'diet_score',
              'sleep_hours_per_day', 'screen_time_hours_per_day', 'bmi',
              'waist_to_hip_ratio', 'systolic_bp', 'diastolic_bp',
              'heart_rate',
              'cholesterol_total', 'hdl_cholesterol', 'ldl_cholesterol',
              'triglycerides', 'gender', 'ethnicity', 'education_level',
              'income_level', 'smoking_status', 'employment_status',
              'family_history_diabetes', 'hypertension_history',
              'cardiovascular_history', 'diagnosed_diabetes'],
              dtype='object')
```

## #### Column Descriptions:

The dataset contains **26 columns** (25 features + 1 target), covering four domains: clinical measurements, lifestyle habits, demographic information and personal medical history.

Note: since the `id` is just a unique row identifier, it carries no predictive value and it's not considered in the following description of the columns. So totally, we have 25 columns (24 features + 1 target) to analyze

## ##### 🏥 Clinical Measurements

| Column                          | Type                 | Description                                                                             |
|---------------------------------|----------------------|-----------------------------------------------------------------------------------------|
| <code>bmi</code>                | <code>float64</code> | Body Mass Index (kg/m <sup>2</sup> ).                                                   |
| <code>waist_to_hip_ratio</code> | <code>float64</code> | Ratio of waist circumference to hip circumference.                                      |
| <code>systolic_bp</code>        | <code>int64</code>   | Systolic blood pressure (mmHg)                                                          |
| <code>diastolic_bp</code>       | <code>int64</code>   | Diastolic blood pressure (mmHg)                                                         |
| <code>heart_rate</code>         | <code>int64</code>   | Resting heart rate (bpm - beats per minute).                                            |
| <code>cholesterol_total</code>  | <code>int64</code>   | Total cholesterol level (mg/dL).                                                        |
| <code>hdl_cholesterol</code>    | <code>int64</code>   | High-Density Lipoprotein (HDL) cholesterol (mg/dL), commonly called "good" cholesterol. |
| <code>ldl_cholesterol</code>    | <code>int64</code>   | Low-Density Lipoprotein cholesterol (mg/dL), commonly called "bad" cholesterol.         |
| <code>triglycerides</code>      | <code>int64</code>   | Triglyceride level (mg/dL), the most common type of fat in the blood.                   |

---

## ##### 🧑 Lifestyle Habits

| Column                                          | Type                 | Description                                                                          |
|-------------------------------------------------|----------------------|--------------------------------------------------------------------------------------|
| <code>alcohol_consumption_per_week</code>       | <code>int64</code>   | Number of alcoholic drinks consumed per week.                                        |
| <code>physical_activity_minutes_per_week</code> | <code>int64</code>   | Total minutes of physical activity per week.                                         |
| <code>diet_score</code>                         | <code>float64</code> | A score reflecting overall dietary quality. Higher scores indicate a healthier diet. |
| <code>sleep_hours_per_day</code>                | <code>float64</code> | Average daily sleep hours.                                                           |
| <code>screen_time_hours_per_day</code>          | <code>float64</code> | Average daily hours spent on screens.                                                |
| <code>smoking_status</code>                     | <code>object</code>  | Smoking behavior ('Never', 'Former', 'Current')                                      |

---

## ##### 👤 Demographic Information

| Column           | Type               | Description                  |
|------------------|--------------------|------------------------------|
| <code>age</code> | <code>int64</code> | Age of the patient in years. |

```
| gender | object | Biological sex of the patient ( 'Male', 'Female', 'Other'). |
| ethnicity | object | Ethnic background ('White', 'Hispanic', 'Black', 'Asian', 'Other'). |
| education_level | object | Highest level of education completed. ('No formal',
'Highschool', 'Graduate', 'Postgraduate') |
| income_level | object | Income of the patient. Likely ordinal ( 'Low' < 'Lower-Middle'
< 'Middle' < 'Upper-Middle' < 'High' ) |
| employment_status | object | Current employment situation ('Employed',
'Unemployed', 'Retired', 'Student'). |
```

---

#### ##### 🏥 Medical History

```
| Column | Type | Description |
|-----|-----|-----|
| family_history_diabetes | int64 | Binary flag (0 = No, 1 = Yes) indicating family
history of diabetes. |
| hypertension_history | int64 | Binary flag (0 = No, 1 = Yes) indicating a diagnosis of
hypertension. |
| cardiovascular_history | int64 | Binary flag (0 = No, 1 = Yes) indicating a history of
cardiovascular disease. |
```

---

#### ##### 🎯 Target Variable

```
| Column | Type | Description |
|-----|-----|-----|
| diagnosed_diabetes | float64 | Binary target variable (0.0 = No diabetes, 1.0 =
Diabetes). This is the variable our models will learn to predict. |
```

Display the first 5 rows to get a feel for the data:

In [6]: `df.head()`

Out[6]:

|   | id | age | alcohol_consumption_per_week | physical_activity_minutes_per_week |
|---|----|-----|------------------------------|------------------------------------|
| 0 | 0  | 31  | 1                            | 45                                 |
| 1 | 1  | 50  | 2                            | 73                                 |
| 2 | 2  | 32  | 3                            | 158                                |
| 3 | 3  | 54  | 3                            | 77                                 |
| 4 | 4  | 54  | 1                            | 55                                 |

5 rows × 26 columns



`df.info()` gives a compact structural summary of the dataframe:

- total number of rows and columns
- each column's name, non-null count, and dtype
- memory usage

In [7]: df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 700000 entries, 0 to 699999
Data columns (total 26 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   id                                         700000 non-null  int64
1   age                                       700000 non-null  int64
2   alcohol_consumption_per_week            700000 non-null  int64
3   physical_activity_minutes_per_week      700000 non-null  int64
4   diet_score                              700000 non-null  float64
5   sleep_hours_per_day                     700000 non-null  float64
6   screen_time_hours_per_day               700000 non-null  float64
7   bmi                                       700000 non-null  float64
8   waist_to_hip_ratio                      700000 non-null  float64
9   systolic_bp                             700000 non-null  int64
10  diastolic_bp                             700000 non-null  int64
11  heart_rate                               700000 non-null  int64
12  cholesterol_total                       700000 non-null  int64
13  hdl_cholesterol                         700000 non-null  int64
14  ldl_cholesterol                         700000 non-null  int64
15  triglycerides                           700000 non-null  int64
16  gender                                   700000 non-null  object
17  ethnicity                               700000 non-null  object
18  education_level                         700000 non-null  object
19  income_level                            700000 non-null  object
20  smoking_status                          700000 non-null  object
21  employment_status                       700000 non-null  object
22  family_history_diabetes                 700000 non-null  int64
23  hypertension_history                   700000 non-null  int64
24  cardiovascular_history                 700000 non-null  int64
25  diagnosed_diabetes                     700000 non-null  float64
dtypes: float64(6), int64(14), object(6)
memory usage: 138.9+ MB
```

**Observations:**

The dataset contains **700,000 rows and 26 columns** with **no missing values** across any feature.

The columns fall into three groups by dtype:

- **Continuous numerical** ( float64 )
- **Discrete numerical** ( int64 )
- **Categorical** ( object )

The **target** `diagnosed_diabetes` is stored as `float64` rather than `int64` . It should contain only `0.0` and `1.0` values (see later).

The total memory footprint is **138.9 MB**, manageable in-memory without any chunking strategy.

Overall check of type and non-unique features:

```
In [8]: pd.DataFrame({
        'dtype': df.dtypes,
        'nunique': df.nunique(),
        'sample': [df[c].dropna().iloc[0] if df[c].notna().any() else None
        for c in df.columns]
    })
```

Out[8]:

|                                    | dtype   | nunique | sample       |
|------------------------------------|---------|---------|--------------|
| id                                 | int64   | 700000  | 0            |
| age                                | int64   | 71      | 31           |
| alcohol_consumption_per_week       | int64   | 9       | 1            |
| physical_activity_minutes_per_week | int64   | 565     | 45           |
| diet_score                         | float64 | 99      | 7.7          |
| sleep_hours_per_day                | float64 | 69      | 6.8          |
| screen_time_hours_per_day          | float64 | 151     | 6.1          |
| bmi                                | float64 | 231     | 33.4         |
| waist_to_hip_ratio                 | float64 | 36      | 0.93         |
| systolic_bp                        | int64   | 71      | 112          |
| diastolic_bp                       | int64   | 54      | 70           |
| heart_rate                         | int64   | 60      | 62           |
| cholesterol_total                  | int64   | 154     | 199          |
| hdl_cholesterol                    | int64   | 69      | 58           |
| ldl_cholesterol                    | int64   | 151     | 114          |
| triglycerides                      | int64   | 228     | 102          |
| gender                             | object  | 3       | Female       |
| ethnicity                          | object  | 5       | Hispanic     |
| education_level                    | object  | 4       | Highschool   |
| income_level                       | object  | 5       | Lower-Middle |
| smoking_status                     | object  | 3       | Current      |
| employment_status                  | object  | 4       | Employed     |
| family_history_diabetes            | int64   | 2       | 0            |
| hypertension_history               | int64   | 2       | 0            |
| cardiovascular_history             | int64   | 2       | 0            |
| diagnosed_diabetes                 | float64 | 2       | 1.0          |

As it's written before the **target** `diagnosed_diabetes` contains only `0.0` and `1.0` values.

Check the **object** features:

```
In [9]: categorical_cols = df.select_dtypes('object').columns.to_list()

# category values with relative frequencies (to spot class imbalance
inside a feature)
for col in categorical_cols:
    print(f"\n \033[1m--- 📄 {col} ({len(df[col].unique())} unique
elements)\033[0m ---")
    print(df[col].value_counts(normalize=True,
dropna=False).mul(100).round(2).astype(str) + ' %')
```

```
🔍[1m--- 📄 gender (3 unique elements)🔍[0m ---
gender
Female    51.89 %
Male      47.58 %
Other      0.53 %
Name: proportion, dtype: object
```

```
🔍[1m--- 📄 ethnicity (5 unique elements)🔍[0m ---
ethnicity
White      55.16 %
Hispanic   18.57 %
Black      15.19 %
Asian      8.59 %
Other       2.49 %
Name: proportion, dtype: object
```

```
🔍[1m--- 📄 education_level (4 unique elements)🔍[0m ---
education_level
Highschool    49.16 %
Graduate      37.32 %
Postgraduate  11.38 %
No formal     2.14 %
Name: proportion, dtype: object
```

```
🔍[1m--- 📄 income_level (5 unique elements)🔍[0m ---
income_level
Middle      41.51 %
Lower-Middle 25.51 %
Upper-Middle 18.26 %
Low         12.26 %
High        2.46 %
Name: proportion, dtype: object
```

```
🔍[1m--- 📄 smoking_status (3 unique elements)🔍[0m ---
smoking_status
Never       70.64 %
Current     14.77 %
Former      14.6 %
Name: proportion, dtype: object
```

```
🔍[1m--- 📄 employment_status (4 unique elements)🔍[0m ---
employment_status
Employed    73.74 %
Retired     16.53 %
Unemployed   7.11 %
```

Student 2.62 %  
 Name: proportion, dtype: object

Find hidden categoricals among the int64 and float64 columns. Many features in this dataset are likely binary flags (family history, etc.)

```
In [10]: #values with relative frequencies (to spot class imbalance inside a
feature)
for col in df.select_dtypes(['int64','float64']).columns.to_list():
    n = df[col].nunique()
    if n <= 20:
        print(f"\n \033[1m--- 📄 {col} ({len(df[col].unique())} unique
elements) --- \033[0m")
        print(df[col].value_counts(normalize=True,
dropna=False).mul(100).round(2).astype(str) + ' %')
```

```
🔍[1m--- 📄 alcohol_consumption_per_week (9 unique elements) --- 🔍[0m
🔍[0m
alcohol_consumption_per_week
2    35.23 %
1    35.19 %
3    19.65 %
4     7.57 %
5     1.9 %
6     0.39 %
7     0.06 %
8     0.01 %
9     0.0 %
Name: proportion, dtype: object
```

```
🔍[1m--- 📄 family_history_diabetes (2 unique elements) --- 🔍[0m
family_history_diabetes
0    85.06 %
1    14.94 %
Name: proportion, dtype: object
```

```
🔍[1m--- 📄 hypertension_history (2 unique elements) --- 🔍[0m
hypertension_history
0    81.8 %
1    18.2 %
Name: proportion, dtype: object
```

```
🔍[1m--- 📄 cardiovascular_history (2 unique elements) --- 🔍[0m
cardiovascular_history
0    96.97 %
1     3.03 %
Name: proportion, dtype: object
```

```
🔍[1m--- 📄 diagnosed_diabetes (2 unique elements) --- 🔍[0m
diagnosed_diabetes
1.0    62.33 %
0.0    37.67 %
Name: proportion, dtype: object
```

Check for any duplicate row:

```
In [11]: print(f"Exact duplicates: {df.drop(columns='id').duplicated().sum()}")
```

Exact duplicates: 0

The **describe** method of pandas dataframes gives a short summary

```
In [12]: df.describe()
```

Out[12]:

|       | id            | age           | alcohol_consumption_per_week |
|-------|---------------|---------------|------------------------------|
| count | 700000.000000 | 700000.000000 | 700000.000000                |
| mean  | 349999.500000 | 50.359734     | 2.072411                     |
| std   | 202072.738554 | 11.655520     | 1.048189                     |
| min   | 0.000000      | 19.000000     | 1.000000                     |
| 25%   | 174999.750000 | 42.000000     | 1.000000                     |
| 50%   | 349999.500000 | 50.000000     | 2.000000                     |
| 75%   | 524999.250000 | 58.000000     | 3.000000                     |
| max   | 699999.000000 | 89.000000     | 9.000000                     |

## 3.2 Clean data

Check for NaN or null values:

```
In [13]: print('NaN count:', df.isnull().sum().sum(), '\n')
df.isnull().sum()
```

NaN count: 0

```
Out[13]: id                                0
         age                                0
         alcohol_consumption_per_week       0
         physical_activity_minutes_per_week  0
         diet_score                          0
         sleep_hours_per_day                 0
         screen_time_hours_per_day           0
         bmi                                 0
         waist_to_hip_ratio                  0
         systolic_bp                         0
         diastolic_bp                       0
         heart_rate                         0
         cholesterol_total                   0
         hdl_cholesterol                     0
         ldl_cholesterol                     0
         triglycerides                       0
         gender                              0
         ethnicity                           0
         education_level                     0
         income_level                        0
         smoking_status                      0
         employment_status                   0
         family_history_diabetes              0
         hypertension_history                 0
         cardiovascular_history               0
         diagnosed_diabetes                   0
         dtype: int64
```

NaN count: 0 across all 26 columns: no rows need to be removed for missing data.  
There aren't any null or nan values

### 3.3 Target Variable Distribution & Class Balance

Before any modeling decision we need to understand how the two classes are distributed.

A strong imbalance would force us to use stratified splits, class-weighting or resampling techniques. A near-balanced target lets us train with standard procedures.

```

In [14]: # Plot class distribution of the target
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# Countplot
counts = df[TARGET].value_counts().sort_index() # Take the counts of
target values
sns.countplot(x=TARGET, data=df, ax=axes[0], palette=['#4C72B0',
'#C44E52']) # Compute the histogram
axes[0].set_title('Class counts')
axes[0].set_xlabel('diagnosed_diabetes')
axes[0].set_ylabel('count')

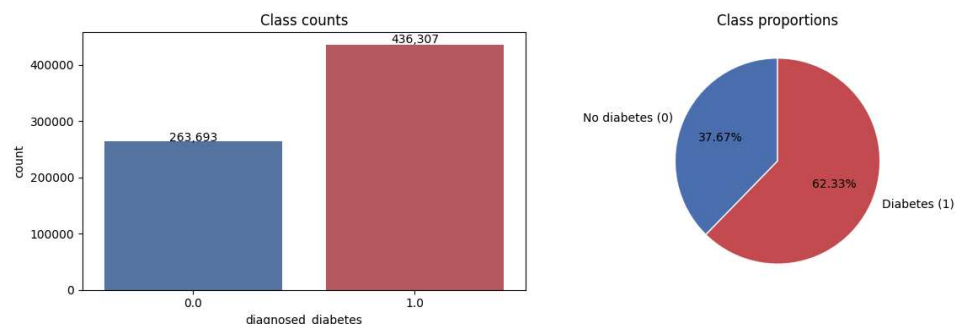
# Text over the columns
no_diagn = 0
yes_diagn = 1
axes[0].text(0, counts.values[no_diagn] + counts.values[no_diagn]*0.005,
f'{counts.values[no_diagn]:,}', ha='center', fontsize=10)
axes[0].text(1, counts.values[yes_diagn] +
counts.values[yes_diagn]*0.005, f'{counts.values[yes_diagn]:,}',
ha='center', fontsize=10)

# Pieplot with percentages
axes[1].pie(counts.values,
            labels=['No diabetes (0)', 'Diabetes (1)'],
            autopct='%1.2f%%',
            colors=['#4C72B0', '#C44E52'],
            startangle=90,
            wedgeprops={'edgecolor': 'white'})
axes[1].set_title('Class proportions')

plt.tight_layout()
plt.show()

ratio = counts[1.0] / counts[0.0]
print(f"0 (no diabetes): {counts[0.0]:.2f}")
print(f"1 (yes diabetes): {counts[1.0]:.2f}")
print(f"Positive/Negative ratio: {ratio:.2f}")

```



```

0 (no diabetes): 263693.00
1 (yes diabetes): 436307.00
Positive/Negative ratio: 1.65

```

**Observations:**

- Class proportions: **62.33%** positives ( 1.0 ), **37.67%** negatives ( 0.0 ).
- Positive / negative ratio: **1.65**.
- Target is **mildly imbalanced**, with the positive class as majority

### 3.4 Numerical Features — Univariate Distributions

We separate truly *continuous* numerical features from *binary* flags (already inspected above) and plot a histogram + KDE for each one to inspect the shape, symmetry, and the presence of suspicious peaks.

```
In [15]: # Continuous numerical features (exclude id, target, binary flags, and
         low-card discretely)
         binary_cols = ['family_history_diabetes', 'hypertension_history',
                        'cardiovascular_history']
         discrete_cols = ['alcohol_consumption_per_week']
         numeric_cols = [c for c in df.select_dtypes(include=['int64',
                        'float64']).columns
                        if c not in ['id', TARGET] + binary_cols +
                        discrete_cols]

         print(f"Continuous numerical features ({len(numeric_cols)}):")
         print(numeric_cols)
```

```
Continuous numerical features (14):
['age', 'physical_activity_minutes_per_week', 'diet_score',
'sleep_hours_per_day', 'screen_time_hours_per_day', 'bmi',
'waist_to_hip_ratio', 'systolic_bp', 'diastolic_bp', 'heart_rate',
'cholesterol_total', 'hdl_cholesterol', 'ldl_cholesterol',
'triglycerides']
```

```

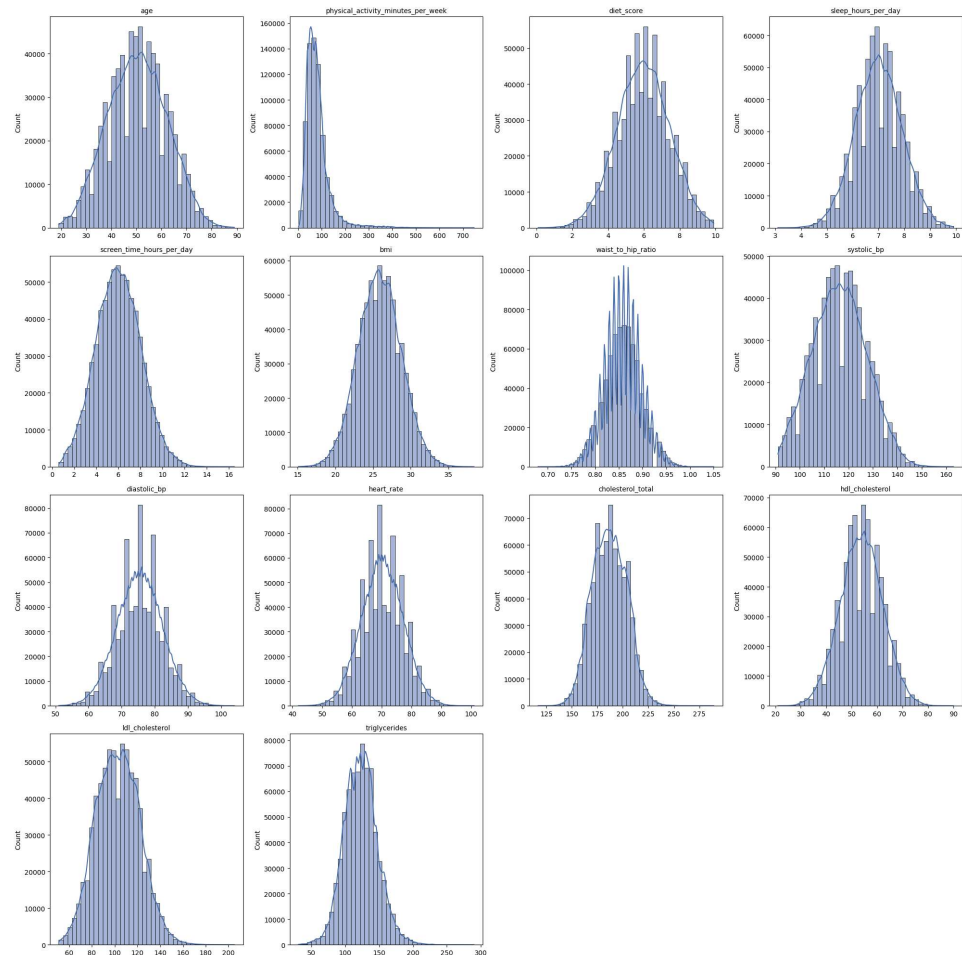
In [16]: # Histograms with KDE for every continuous numerical feature
n_cols = 4
n_rows = 4
fig, axes = plt.subplots(n_rows, n_cols, figsize=(20, 20))
axes = axes.flatten()

for i, col in enumerate(numeric_cols):
    sns.histplot(df[col], kde=True, ax=axes[i], color='#4C72B0',
bins=40)
    axes[i].set_title(col, fontsize=10)
    axes[i].set_xlabel('')

# Hide unused axes
for j in range(len(numeric_cols), len(axes)):
    axes[j].axis('off')

plt.tight_layout()
plt.show()

```



**Observation:**

- Most clinical features ( bmi , systolic\_bp , diastolic\_bp , heart\_rate , cholesterol\_total , hdl\_cholesterol , ldl\_cholesterol , triglycerides , waist\_to\_hip\_ratio ) are roughly **bell-shaped**.
- physical\_activity\_minutes\_per\_week shows a **strong right-skew** with a long tail (a few people training >250 min/week).
- diet\_score , sleep\_hours\_per\_day , screen\_time\_hours\_per\_day look roughly symmetric.
- age is distributed between 19 and 89.

### 3.5 Outlier Detection (Boxplots + IQR rule)

We use the **interquartile range (IQR) rule**: a point is flagged as an outlier when it falls outside  $[Q1 - 1.5 \cdot IQR, Q3 + 1.5 \cdot IQR]$  . The boxplots provide a visual confirmation;

the table quantifies how many points are flagged per feature.

```

In [17]: # Boxplots verticali per ogni feature numerica continua
n_cols = 4
n_rows = 4
plt.figure(figsize=(20, 20))

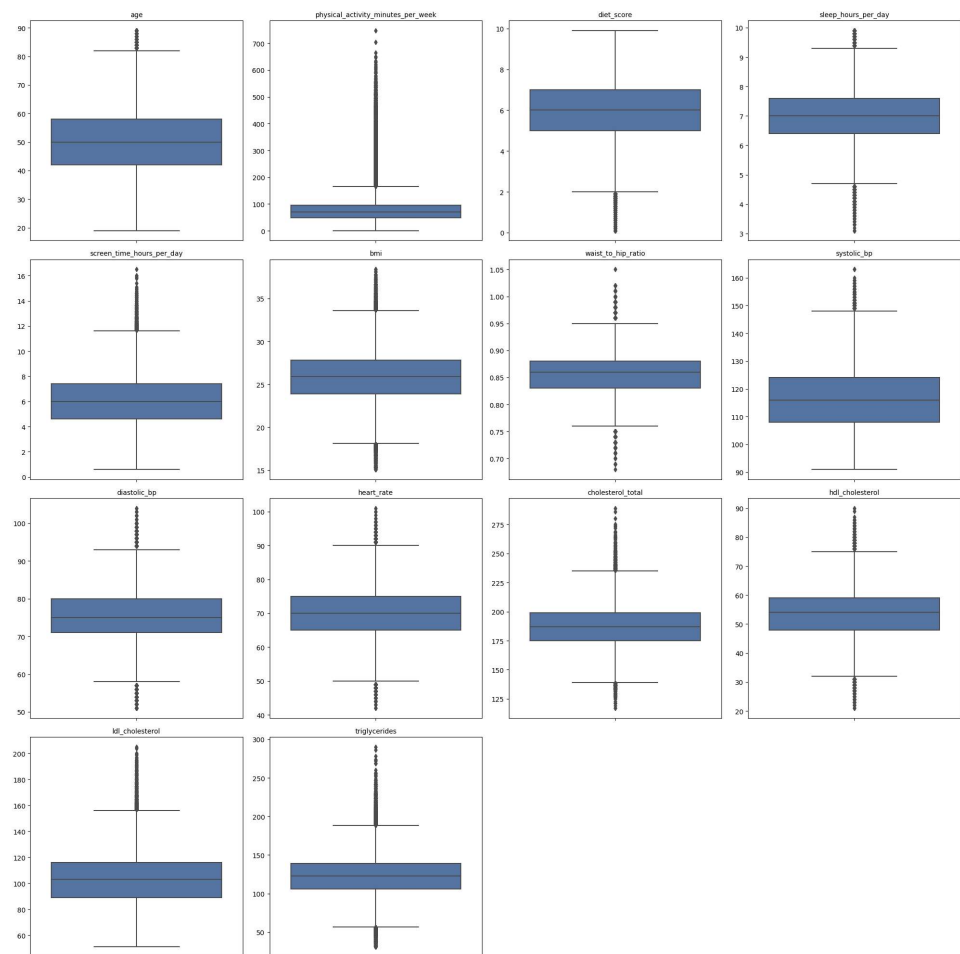
for i, col in enumerate(numeric_cols):
    plt.subplot(n_rows, n_cols, i + 1)

    sns.boxplot(y=df[col], color='#4C72B0')

    plt.title(col, fontsize=10)
    plt.ylabel('')

plt.tight_layout()
plt.show()

```



```
In [18]: # IQR-based outlier detection (pandas .quantile())
Q1 = df[numeric_cols].quantile(0.25) #
Q3 = df[numeric_cols].quantile(0.75)
IQR = Q3 - Q1

# Boundaries
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# outliers per column
outlier_mask = (df[numeric_cols] < lower_bound) | (df[numeric_cols] >
upper_bound)
n_outliers = outlier_mask.sum()
pct_outliers = (n_outliers / len(df) * 100).round(2)

max_outlier = df[numeric_cols].where(outlier_mask).max()

iqr_summary = pd.DataFrame({
    'Q1': Q1.round(2),
    'Q3': Q3.round(2),
    'IQR': IQR.round(2),
    'lower_bound': lower_bound.clip(lower=0).round(2),
    'upper_bound': upper_bound.round(2),
    'max_outlier': max_outlier,
    'n_outliers': n_outliers,
    'pct_outliers': pct_outliers,
}).sort_values('n_outliers', ascending=False)

iqr_summary
```

Out[18]:

|                                    | Q1     | Q3     | IQR   | lower_boun |
|------------------------------------|--------|--------|-------|------------|
| physical_activity_minutes_per_week | 49.00  | 96.00  | 47.00 | 0.00       |
| triglycerides                      | 106.00 | 139.00 | 33.00 | 56.50      |
| waist_to_hip_ratio                 | 0.83   | 0.88   | 0.05  | 0.75       |
| sleep_hours_per_day                | 6.40   | 7.60   | 1.20  | 4.60       |
| diastolic_bp                       | 71.00  | 80.00  | 9.00  | 57.50      |
| hdl_cholesterol                    | 48.00  | 59.00  | 11.00 | 31.50      |
| bmi                                | 23.90  | 27.80  | 3.90  | 18.05      |
| screen_time_hours_per_day          | 4.60   | 7.40   | 2.80  | 0.40       |
| ldl_cholesterol                    | 89.00  | 116.00 | 27.00 | 48.50      |
| diet_score                         | 5.00   | 7.00   | 2.00  | 2.00       |
| heart_rate                         | 65.00  | 75.00  | 10.00 | 50.00      |
| cholesterol_total                  | 175.00 | 199.00 | 24.00 | 139.00     |
| age                                | 42.00  | 58.00  | 16.00 | 18.00      |

|             | Q1     | Q3     | IQR   | lower_boun |
|-------------|--------|--------|-------|------------|
| systolic_bp | 108.00 | 124.00 | 16.00 | 84.00      |

### Observations (from box-plots + IQR):

- **Heavy upper tails** (max well above  $Q3 + 1.5 \cdot IQR$ ):
  - physical\_activity\_minutes\_per\_week :  $Q3 = 96$ , upper bound  $\approx 166$ , **max = 747** (extreme outliers).
  - triglycerides :  $Q3 = 139$ , upper  $\approx 189$ , **max = 290**.
  - cholesterol\_total :  $Q3 = 199$ , upper  $\approx 235$ , **max = 289**.
  - ldl\_cholesterol :  $Q3 = 116$ , upper  $\approx 157$ , **max = 205**.
  - screen\_time\_hours\_per\_day :  $Q3 = 7.4$ , upper  $\approx 11.6$ , **max = 16.5**.
  - systolic\_bp : upper  $\approx 148$ , **max = 163**.
  - bmi : upper  $\approx 33.7$ , **max = 38.4**.
- **Two-sided tails:** hdl\_cholesterol , waist\_to\_hip\_ratio , heart\_rate .
- Implication for preprocessing: linear models will benefit from a **scaler** to limit the influence of these tails; tree-based models can use the unscaled values because these type of models (Random Forest, XGBoost, ...) are less affected by extreme points

## 3.6 Categorical Features vs Target

Countplots with `hue=TARGET` reveal whether the diabetes rate varies across the levels of each categorical / binary feature. Differences in within-category proportions are a first indication of predictive value.

```
In [19]: # Combine pure categoricals with the binary medical flags
cat_like = df.select_dtypes('object').columns.tolist() + binary_cols +
['alcohol_consumption_per_week']
print(f"Categorical-like features ({len(cat_like)}):", cat_like)
```

```
Categorical-like features (10): ['gender', 'ethnicity',
'education_level', 'income_level', 'smoking_status',
'employment_status', 'family_history_diabetes',
'hypertension_history', 'cardiovascular_history',
'alcohol_consumption_per_week']
```

```

In [20]: n_cols = 3
n_rows = 4
plt.figure(figsize=(20, 20))

for i, col in enumerate(cat_like):

    plt.subplot(n_rows, n_cols, i + 1)

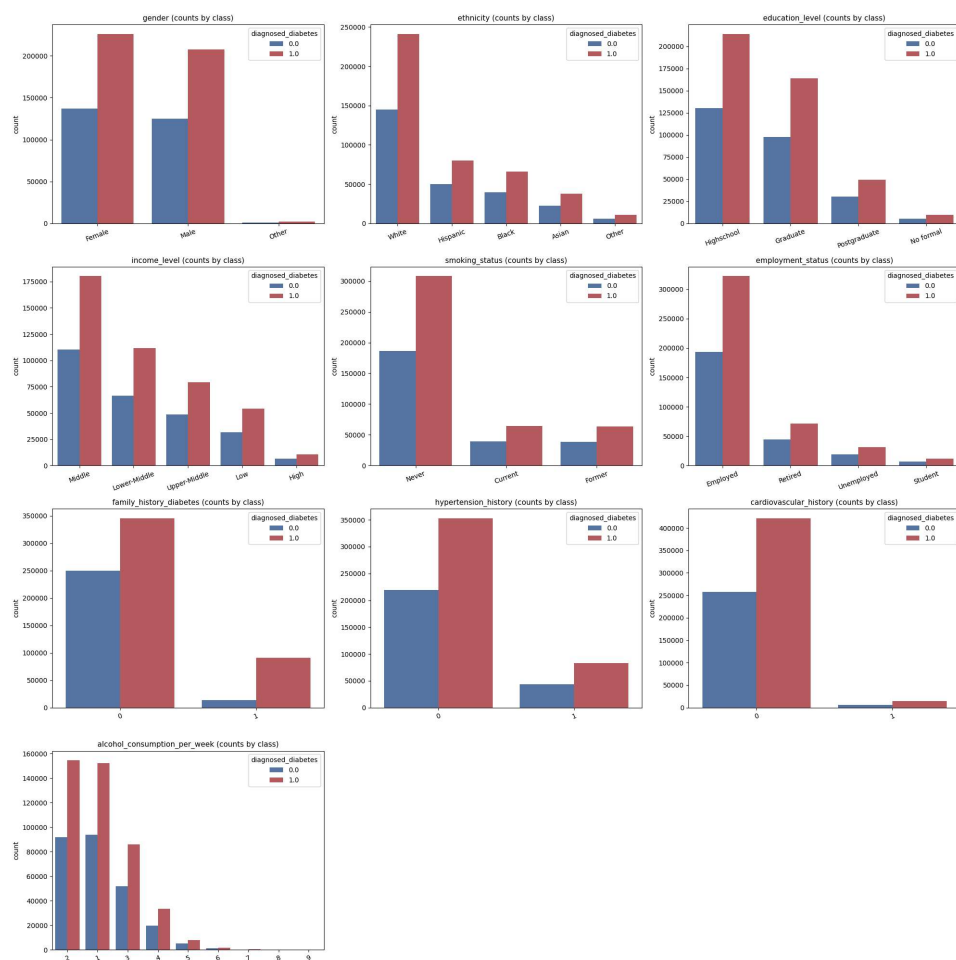
    order = df[col].value_counts().index

    sns.countplot(x=col, hue=TARGET, data=df, order=order,
                  palette=['#4C72B0', '#C44E52'])

    plt.title(f'{col} (counts by class)', fontsize=11)
    plt.xlabel('')
    plt.xticks(rotation=20)

plt.tight_layout()
plt.show()

```



### Observations:

- The **positive class (1)** dominates every category, consistent with the global **62%/38%** target split.
- alcohol\_consumption\_per\_week : concentrated on **values 1, 2, 3**;
- Binary medical flags are heavily skewed toward 0: family\_history\_diabetes , hypertension\_history , cardiovascular\_history .

### 3.7 Numerical Features by Target Class

To see whether continuous features separate the two classes, we plot a **KDE per class** for each feature. Features whose distributions are visibly shifted between class 0 and class 1 carry direct predictive signal.

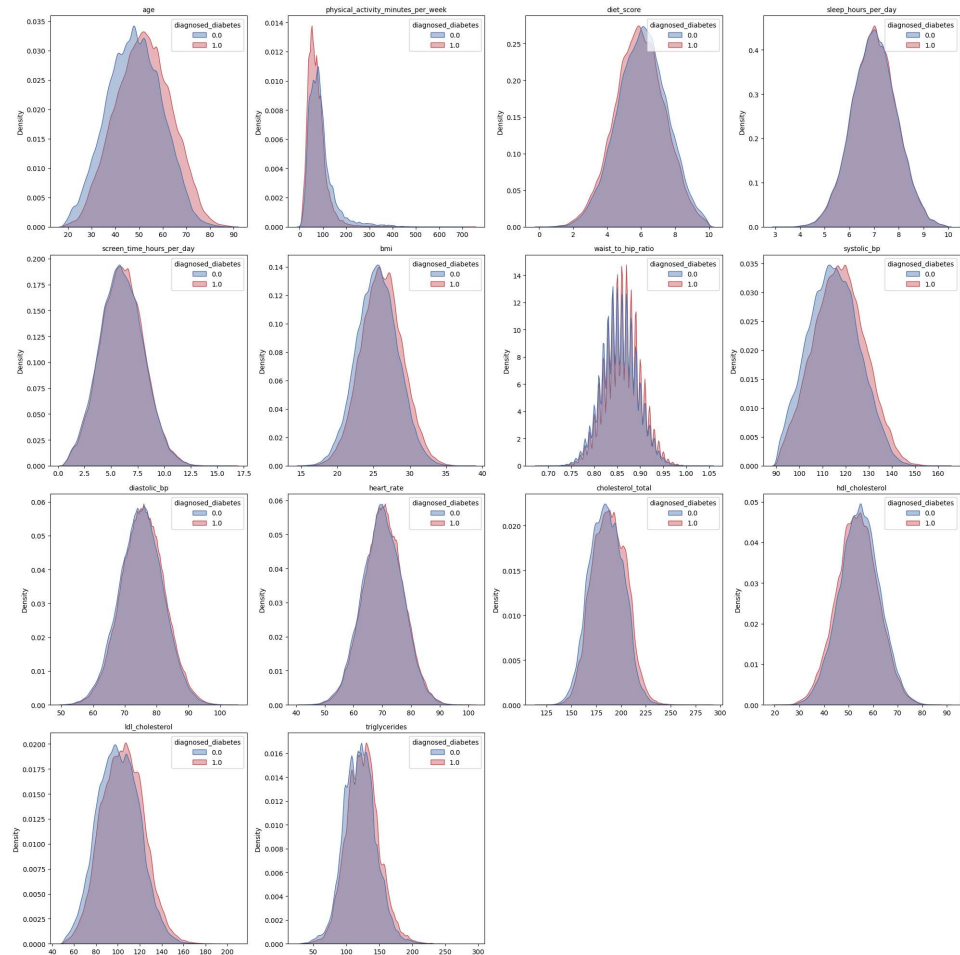
```
In [21]: # KDE plot per class for each continuous numerical feature
n_cols = 4
n_rows = 4
plt.figure(figsize=(20, 20))

for i, col in enumerate(numeric_cols):
    # Attiva il subplot corrispondente (indice 1-based)
    plt.subplot(n_rows, n_cols, i + 1)

    # Seaborn disegna nel subplot corrente
    sns.kdeplot(data=df, x=col, hue=TARGET,
                common_norm=False, fill=True, alpha=0.4,
                palette=['#4C72B0', '#C44E52'])

    # Formattazione
    plt.title(col, fontsize=10)
    plt.xlabel('')

plt.tight_layout()
plt.show()
```



## 3.8 Correlation Analysis

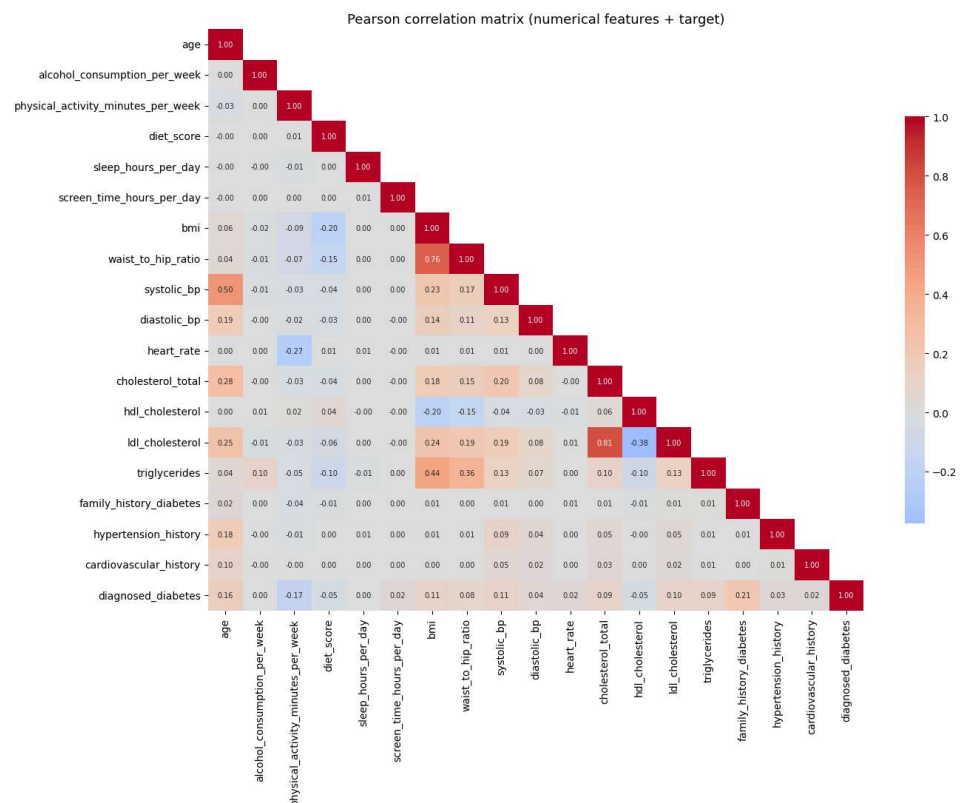
A correlation heatmap helps detect:

- **redundant features** (very high pairwise correlations → multicollinearity),
- **direct signals to the target** (which feature correlates most with `diagnosed_diabetes` ).

```
In [22]: # Pearson correlation matrix for numeric features (binary flags + target included)
num_for_corr = [c for c in df.select_dtypes(include=['int64',
'float64']).columns
                if c != 'id']
corr = df[num_for_corr].corr()

plt.figure(figsize=(14, 11))
mask = np.triu(np.ones_like(corr, dtype=bool), k=1)
sns.heatmap(corr, mask=mask, cmap='coolwarm', center=0,
            annot=True, fmt='.2f', annot_kws={'size': 7},
            cbar_kws={'shrink': 0.7})
plt.title('Pearson correlation matrix (numerical features + target)',
          fontsize=13)
plt.tight_layout()
plt.show()
```

```
/usr/local/lib/python3.11/dist-
packages/matplotlib/colormaps.py:721: RuntimeWarning: invalid
value encountered in less
  xa[xa < 0] = -1
```



```
In [23]: target_corr = (corr[TARGET]
                        .drop(TARGET)
                        .sort_values(key=abs, ascending=True))

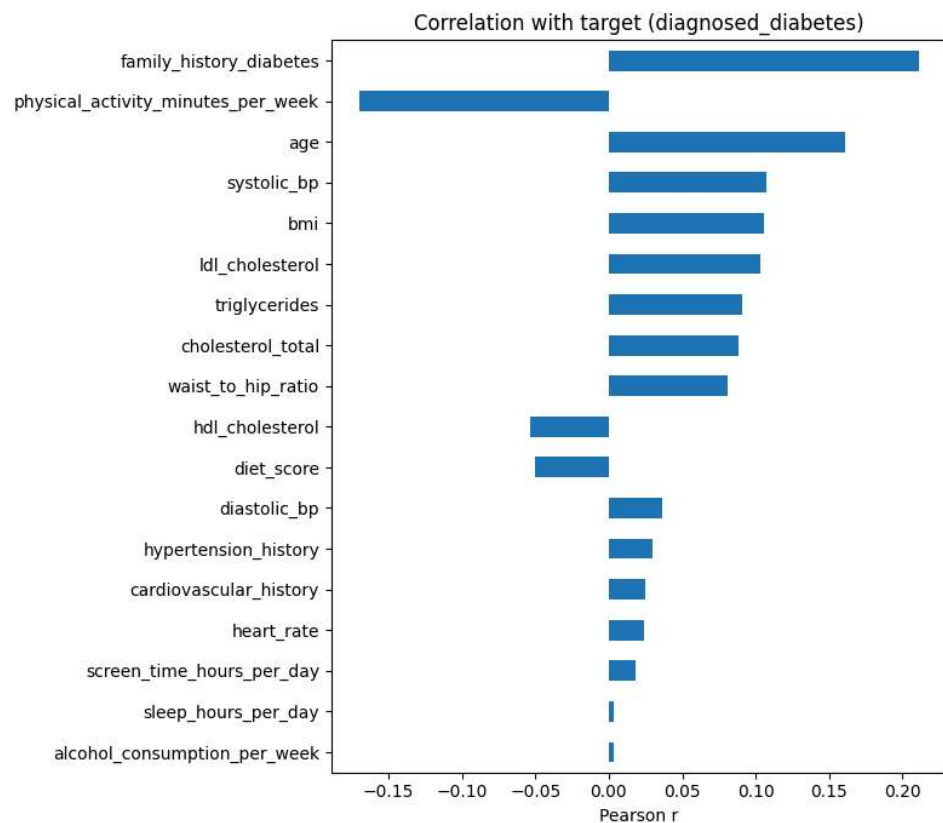
# Inizializza la figura
plt.figure(figsize=(8, 7))

# Crea il subplot singolo (1 riga, 1 colonna, indice 1)
plt.subplot(1, 1, 1)

# Il metodo plot di Pandas userà automaticamente il subplot attivo
target_corr.plot(kind='barh')

# Formattazione tramite l'interfaccia pyplot
plt.title(f'Correlation with target ({TARGET})')
plt.xlabel('Pearson r')

plt.tight_layout()
plt.show()
```



```
In [24]: target_corr.round(3)
```

```
Out[24]: alcohol_consumption_per_week      0.003
         sleep_hours_per_day               0.004
         screen_time_hours_per_day         0.018
         heart_rate                       0.024
         cardiovascular_history            0.025
         hypertension_history              0.030
         diastolic_bp                     0.036
         diet_score                       -0.050
         hdl_cholesterol                  -0.053
         waist_to_hip_ratio                0.081
         cholesterol_total                 0.088
         triglycerides                     0.091
         ldl_cholesterol                  0.103
         bmi                              0.106
         systolic_bp                      0.107
         age                              0.161
         physical_activity_minutes_per_week -0.170
         family_history_diabetes           0.211
         Name: diagnosed_diabetes, dtype: float64
```

### Observations (correlation with target):

- **All correlations are weak:** the strongest is `family_history_diabetes` .
- **Top positive drivers:** `family_history_diabetes` , `age` , `systolic_bp` , `bmi` , `ldl_cholesterol` , `triglycerides` , `cholesterol_total` , `waist_to_hip_ratio` .
- **Top negative drivers:** `physical_activity_minutes_per_week` , `hdl_cholesterol` , `diet_score` .
- **Near-zero correlation:** `sleep_hours_per_day` , `alcohol_consumption_per_week` .
- The binary flags `hypertension_history` and `cardiovascular_history` are surprisingly low individually.

## 4. Preprocessing Dataset

---

Based on the EDA conclusions, this section transforms the raw dataframe into a **model-ready** matrix.

Pipeline:

1. Drop `id` (non-predictive) and `TARGET` column.
2. Encode categorical variables (ordinal where applicable, one-hot otherwise).
3. Apply a robust scaler to continuous features (saved separately — tree models will use the unscaled version).

## 4.1 Drop the `id` and `TARGET` column

`id` is a unique row identifier with no predictive value. We keep it aside in case it is needed for joins later, but it must not enter the feature matrix.

```
In [25]: print(f"Before dropping id: {df.shape}")
X = df.drop(columns=['id', TARGET]).copy()
print(f"After dropping id: {X.shape}")
```

Before dropping id: (700000, 26)

After dropping id: (700000, 24)

## 4.2 Encoding of categorical features

- **Ordinal encoding** for variables with a clear order:
  - `education_level` : *No formal < Highschool < Graduate < Postgraduate*
  - `income_level` : *Low < Lower-Middle < Middle < Upper-Middle < High*
- **One-hot encoding** for purely nominal variables:
  - `gender` , `ethnicity` , `smoking_status` , `employment_status`

The binary medical flags ( `family_history_diabetes` , `hypertension_history` , `cardiovascular_history` ) are already 0/1 and require no encoding.

```
In [26]: # Ordinal encoding
ordinal_cols = ['education_level', 'income_level']
ordinal_categories = [
    ['No formal', 'Highschool', 'Graduate', 'Postgraduate'], #
    education_level
    ['Low', 'Lower-Middle', 'Middle', 'Upper-Middle', 'High'], #
    income_level
]

ord_enc = OrdinalEncoder(categories=ordinal_categories, dtype=np.int8)
X[ordinal_cols] = ord_enc.fit_transform(X[ordinal_cols])

# One-hot encoding
nominal_cols = ['gender', 'ethnicity', 'smoking_status',
                'employment_status']

ohe = OneHotEncoder(drop=None, sparse_output=False, dtype=np.int8)
ohe_array = ohe.fit_transform(X[nominal_cols])
ohe_columns = ohe.get_feature_names_out(nominal_cols)

df_ohe = pd.DataFrame(ohe_array, columns=ohe_columns, index=X.index)
X = pd.concat([X.drop(columns=nominal_cols), df_ohe], axis=1)

print(f"Shape after encoding: {X.shape}")
X.dtypes.value_counts()
```

Shape after encoding: (700000, 35)

```
Out[26]: int8      17
         int64     13
         float64    5
         Name: count, dtype: int64
```

## 4.3 Dataset split & validation strategy

To ensure a robust evaluation the complete dataset (totaling **700,000 rows**) is managed according to the following splitting and validation architecture:

### #### 1. Initial Split (Hold-Out Method)

The primary dataset is partitioned into two distinct subsets:

- \* **Training Set (~67% - 469K rows):** Used exclusively for model training, hyperparameter tuning, and architecture selection.
- \* **Test Set (~23% - 231K rows):** Kept completely isolated. It will be touched **only once at end** for the evaluation, accurately simulating production behavior on unseen data.

### #### 2. Validation & Model Selection (Cross-Validation)

To leverage the training data efficiently without overfitting during the exploration phase (during `GridSearchCV`), a **5-fold Stratified K-Fold Cross-Validation** strategy is applied directly to the Training Set:

- \* The training data is dynamically split into 5 folds, preserving the original class distribution (stratification).
- \* Iteratively, 4 folds are used for fitting the model, while the remaining fold acts as a temporary **Validation Set**.
- \* It is particularly useful for classification problems in which the class labels are not evenly distributed (i.e data is imbalanced)
- \* It uses stratification which allows original distribution of each class to be maintained across each fold.

```
In [27]: y = df[TARGET]
         print(y.shape)
```

```
(700000,)
```

```
In [28]: X_train, X_test, y_train, y_test = train_test_split(X, y, random_state =
         RANDOM_STATE, train_size = TRAIN_SIZE)
         print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
```

```
(469000, 35) (231000, 35) (469000,) (231000,)
```

## 4.4 StandarScaler

Value of features may have different ranges and units. This variation can impact negatively on the performance of algorithms like KNN, SVM, ... To avoid this issue feature scaling is used to standardize data. In this notebook, we'll use StandarScaler technique: StandardScaler is a feature scaling technique which follows Standard Normal Distribution (SND) and is used to standardize the values of numeric features. It transforms data so that the mean becomes 0 and the standard deviation becomes 1. It's ideal for algorithms like SVM, logistic regression or neural networks that assume data is normally distributed

```
In [ ]: # Initialize the scaler and fit only on the training set
        scaler = StandardScaler()
        X_train[numeric_cols] = scaler.fit_transform(X_train[numeric_cols])

        # Apply the same transformation to the test set (only transform, no fit)
        X_test[numeric_cols] = scaler.transform(X_test[numeric_cols])

        X_train.head()
```

|        | age       | alcohol_consumption_per_week | physical_activity_mil |
|--------|-----------|------------------------------|-----------------------|
| 547157 | -0.200786 | 2                            | -0.414222             |
| 377798 | -1.831513 | 5                            | -1.158364             |
| 155574 | -1.316547 | 1                            | -0.570884             |
| 259218 | 0.914975  | 2                            | 0.545328              |
| 646149 | -0.973235 | 2                            | 1.681123              |

5 rows × 35 columns



## 5. Models & Parameters Definition

### 5.1 Define models and parameters

Prepare the hyperparameter ranges for the modules and define the models for the grid search

```

In [ ]: model_lbls = ['dt'      # decision tree
                    , 'nb'      # gaussian naive bayes
                    #          , 'lp'      # linear perceptron
                    #          , 'svc'     # support vector
                    , 'knn'     # k nearest neighbours
                    , 'adb'     # adaboost
                    , 'rf'      # random forest
                    , 'xgb'     # xgboost
                    , 'lgbm'    # lightgbm
                    , 'catb'    # catboost
                    ]

models = {
    'dt': {'name': 'Decision Tree      ',
           'estimator':
DecisionTreeClassifier(random_state=RANDOM_STATE),
           'param': [{'max_depth': [*range(1,20)],
                       'class_weight':[None,'balanced']}]},
    },
    'nb': {'name': 'Gaussian Naive Bayes',
           'estimator': GaussianNB(),
           'param': [{'var_smoothing': [10**exp for exp in
range(-3,-13,-1)]]}
    },
    'lp': {'name': 'Linear Perceptron   ',
           'estimator': Perceptron(random_state=RANDOM_STATE),
           'param': [{'early_stopping': [True,False],
                       'class_weight':[None,'balanced']}]},
    },
    'svc': {'name': 'Support Vector      ',
            'estimator': SVC(random_state=RANDOM_STATE),
            'param': [{'kernel': ['rbf'],
                        'gamma': [1e-3, 1e-4],
                        'C': [1, 10, 100],
                        },
                      {'kernel': ['linear'],
                        'C': [1, 10, 100],
                        }
            ],
    },
    'knn': {'name': 'K Nearest Neighbor ',
            'estimator': KNeighborsClassifier(),
            'param': [{'n_neighbors': list(range(1,7))}]
    },
    'adb': {'name': 'AdaBoost            ',
            'estimator': AdaBoostClassifier(random_state=RANDOM_STATE),
            'param': [{'n_estimators':[10,20,30,40,50]
                        , 'learning_rate':[0.2,0.5,1,1.5]}]
    },
    'rf': {'name': 'Random forest       ',
           'estimator':
RandomForestClassifier(random_state=RANDOM_STATE),
           'param': [{'max_depth': [*range(4,10)]
                       , 'n_estimators':[*range(10,60,10)]]}
    },
    'xgb': {'name': 'XGboost            ',
            'estimator': XGBClassifier(random_state=RANDOM_STATE,

```

```

objective='binary:logistic'),
    'param': [{ 'n_estimators': [300, 600]
                  , 'learning_rate': [0.02, 0.05, 0.1]
                  , 'min_child_weight': [1, 5, 10]
                  , 'max_depth': [3, 6, 10]
                }]
    },
    'lgbm': { 'name': 'LGBMClassifier',
              'estimator': lgb.LGBMClassifier(objective="binary",
metric="auc", random_state=RANDOM_STATE),
              'param': [{
                  "n_estimators": [100, 200, 300],
                  "learning_rate": [0.02, 0.05, 0.1],
                  "num_leaves": [31, 63, 127],
                  "max_depth": [-1, 3]
                }]
    },
    'catb': { 'name': 'CatBoostClassifier',
              'estimator': CatBoostClassifier(loss_function='Logloss',
random_state=RANDOM_STATE, verbose = 0),
              'param': [{
                  'iterations': [100, 200, 400],
                  'learning_rate': [0.01, 0.1, 0.2],
                  'depth': [3, 6],
                  'auto_class_weights' : ['None', 'Balanced']
                }]
    }
}

```

## 5.2 Set the score functions

Define the scoring metric(s) used to evaluate and rank the models during the grid search. Here we optimise for ROC AUC, which is robust to class imbalance and well suited to the binary target; the commented-out line above lists alternative metrics (accuracy, precision, recall, F1) that can be enabled when needed.

```

In [31]: #scorings =
          ['accuracy', 'precision_macro', 'recall_macro', 'f1_macro', 'roc_auc']
          scorings = ['roc_auc']

```

Number of parameters' combination to explore during grid search

```
In [32]: trials = {lbl: len(list(ParameterGrid(models[lbl]['param']))) for lbl in
models.keys()}
trials
```

```
Out[32]: {'dt': 38,
          'nb': 10,
          'lp': 4,
          'svc': 9,
          'knn': 6,
          'adb': 20,
          'rf': 30,
          'xgb': 54,
          'lgbm': 54,
          'catb': 36}
```

### 5.3 Define the result dataframe

Define two structure to store all the data during the training:

- Prepare an empty list `clfs` to store all the fitted models
- Prepare an empty DataFrame which will collect the results of the fittings with each combination of parameters. The dataframe columns are  
'scoring', 'model', 'best\_params', 'accuracy', 'precision\_macro', 'recall\_macro', 'f1\_ma

```
In [33]: clfs = []
results = pd.DataFrame(columns=
['scoring', 'model', 'best_params'#, 'fit+score_time'

, 'accuracy', 'precision_macro', 'recall_macro', 'f1_macro', 'roc_auc'])
```

## 6. Model Training

The function `GridSearchCV` iterates a cross validation experiment to train and test a model with different combinations of parameter values

- for each parameter is set a list of values to test, `ParametersGrid` will be implicitly called to generate all the combinations
- choose a *score function* which will be used for the optimization. In this case, it is used `roc_auc_score` but there are other solutions like `accuracy_score` , ...
- the output is a dataframe containing
- the set of parameters maximising the score
- the score used for optimisation and all the test scores

## Parameters to collect

`classification_report` produces a dictionary containing some classification performance measures, given the *ground truth* and the *predictions* (use the parameter `output_dict=True` ).

`roc_auc_score` computes the **Area Under the ROC Curve (AUC)** from prediction scores. A value of `1.0` is a perfect classifier and `0.5` is random guessing.

## 6.1 Loop

- repeat for all the chosen scorings
- repeat for all the chosen classification models
- store in `clf` the initialisation of `GridSearchCV` with the appropriate
  - classification model
  - parameters ranges
  - scoring
  - cross validation method `cv` (the same for all)
- fit `clf` with the *train* part of `x` and `y`
- append `clf` to `clfs`
- store in `y_pred` the prediction for the *test* part of `x`
- append `y_pred` to `y_preds`
- store in variable `cr` the `classification_report` produced with the test part of `y` and `y_pred`
- store in `y_scores` the prediction expressed as probability for class predictions for the *test* part of `x`
- store in variable `roc_auc` the score produced by `roc_auc_score` function
- store in the last row of `results` a list containing:
  - the name of the model
  - the `.best_params_` of `clf`
  - a selection of the contents of `cr` and `roc_auc` score
  - 'accuracy',
  - 'macro avg"precision'
  - 'macro avg"recall'
  - 'macro avg"f1-score'
  - 'roc auc'

```

In [ ]: skf = StratifiedKFold(n_splits=N_FOLDS, random_state=RANDOM_STATE,
                             shuffle=True)

for scoring in scorings:
    print(f"Scoring: {scoring}")
    for m in model_lbls:
        print(f"model: {m}")
        clf = GridSearchCV(models[m]['estimator'], models[m]['param'],
cv=skf,
                                scoring = scoring,
                                )

        clf.fit(X_train, y_train)
        clfs.append(clf)
        y_true, y_pred = y_test, clf.predict(X_test)
        # y_preds.append(y_pred)

        # Classification report
        cr = classification_report(y_true,y_pred, output_dict=True
                                , zero_division=1
                                )

        # Scores for ROC AUC
        y_scores = clf.predict_proba(X_test)[: , 1]

        # Compute ROC AUC
        roc_auc = roc_auc_score(y_true, y_scores)

        #roc_auc = roc_auc_score(y_true,y_pred)
        results.loc[len(results)] = [scoring,models[m]
['name'],clf.best_params_
                                # ,
(cclf.cv_results_['mean_fit_time'].sum()+clf.cv_results_['mean_score_time
'].sum())*n_splits
                                ,cr['accuracy']
                                ,cr['macro avg']['precision']
                                ,cr['macro avg']['recall']
                                ,cr['macro avg']['f1-score']
                                ,roc_auc]

```

## 7. Evaluation & Comparison

In this chapter the results collected previously are shown in order to visualize the best model, parameters and analyze the predictions over the test set

## 7.1 Models results

For each scoring show the ranking of the models

For each scoring:

- set a `scoring_filter`
- filter the results of that scoring
- display the filtered dataframe with the `display()` function

```
In [35]: for score in scorings:
        scoring_filter = score
        display(results[results.scoring==scoring_filter]\
                .sort_values(by=scoring_filter,ascending=False)\
                .drop('scoring',axis=1)\
                .style.format(precision=3)\
                .set_caption('Results for scoring "
{}"'.format(scoring_filter))
        )
```

Results for scoring "roc\_auc"

|   | model                | best_params                                                                          | accuracy | precision_macro |
|---|----------------------|--------------------------------------------------------------------------------------|----------|-----------------|
| 7 | CatBoostClassifier   | {'auto_class_weights': 'None', 'depth': 6, 'iterations': 400, 'learning_rate': 0.2}  | 0.684    | 0.662           |
| 6 | LGBMClassifier       | {'learning_rate': 0.1, 'max_depth': -1, 'n_estimators': 300, 'num_leaves': 31}       | 0.683    | 0.660           |
| 5 | XGboost              | {'learning_rate': 0.05, 'max_depth': 6, 'min_child_weight': 10, 'n_estimators': 600} | 0.684    | 0.661           |
| 3 | AdaBoost             | {'learning_rate': 1.5, 'n_estimators': 50}                                           | 0.667    | 0.641           |
| 4 | Random forest        | {'max_depth': 9, 'n_estimators': 50}                                                 | 0.660    | 0.646           |
| 0 | Decision Tree        | {'class_weight': None, 'max_depth': 9}                                               | 0.662    | 0.633           |
| 1 | Gaussian Naive Bayes | {'var_smoothing': 0.001}                                                             | 0.625    | 0.616           |
| 2 | K Nearest Neighbor   | {'n_neighbors': 6}                                                                   | 0.576    | 0.560           |

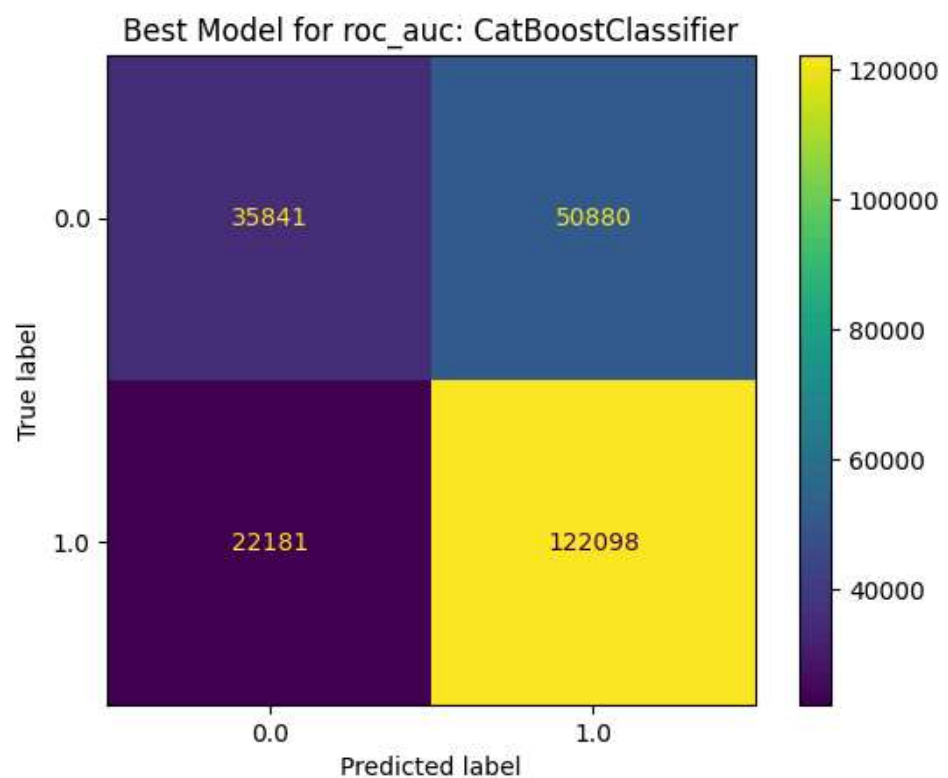
## 7.2 Confusion matrix

Use the `ConfusionMatrixDisplay` with the best model of each scoring to compare the predictions

Repeat for every scoring:

- filter the results for the current scoring
- find the row with the best value of the scoring
- display the confusion matrix with an appropriate title

```
In [36]: for score in scorings:
          scoring_filter = score
          # bests[score] =
          results.loc[results.scoring==scoring_filter,scoring_filter].idxmax(axis=
          0)
          best_row =
          results.loc[results.scoring==scoring_filter,scoring_filter].idxmax(axis=
          0)
          disp = ConfusionMatrixDisplay.from_estimator(X=X_test, y=y_test,
          estimator = clfs[best_row])
          # disp.ax_.set_title("Best Model for {}:".
          {}.format(score,results.at[bests[score],'model']))
          disp.ax_.set_title("Best Model for {}:".
          {}.format(score,results.at[best_row,'model']))
```



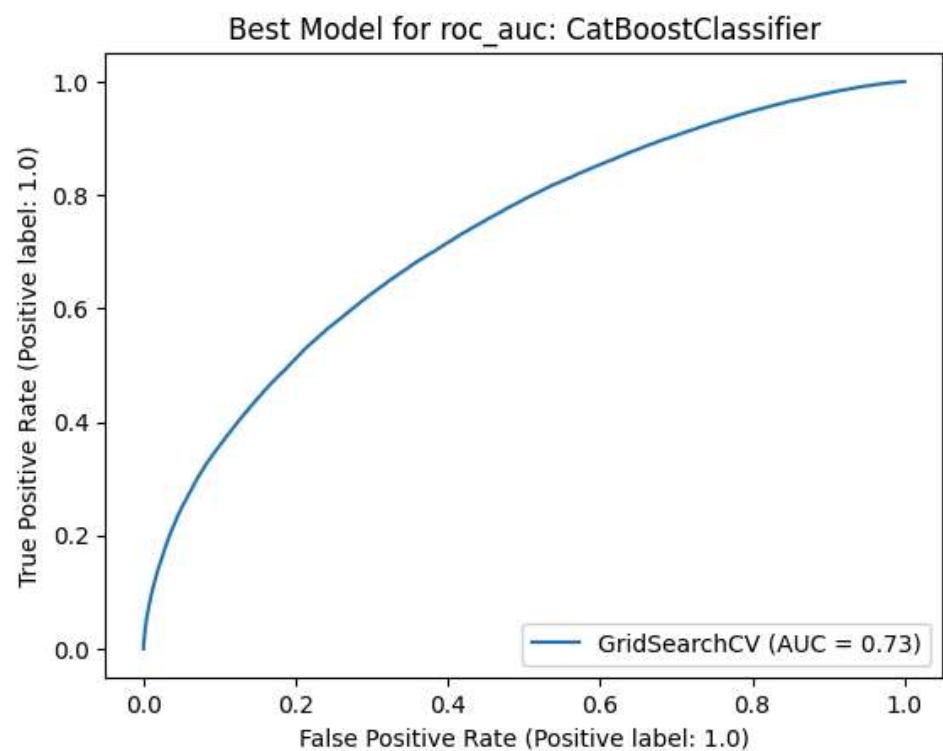
## 7.3 Roc Curve

Use the `RocCurveDisplay` with the best model of each scoring to compare the predictions

Repeat for every scoring:

- filter the results for the current scoring
- find the row with the best value of the scoring
- display the roc curve with an appropriate title

```
In [37]: for score in scorings:
        scoring_filter = score
        # bests[score] =
        results.loc[results.scoring==scoring_filter,scoring_filter].idxmax(axis=
        0)
        best_row =
        results.loc[results.scoring==scoring_filter,scoring_filter].idxmax(axis=
        0)
        disp = RocCurveDisplay.from_estimator(X=X_test, y=y_test, estimator
        = clfs[best_row])
        # disp.ax_.set_title("Best Model for {}:
        {}".format(score,results.at[bests[score],'model']))
        disp.ax_.set_title("Best Model for {}:
        {}".format(score,results.at[best_row,'model']))
```



## 8. Conclusions

---

This notebook implemented a complete, end-to-end machine learning pipeline for the binary classification task of predicting a diabetes diagnosis (from exploratory analysis and cleaning, through preprocessing and encoding, to the training, tuning and comparison of eight different classifiers).

### 8.1 Model comparison

All models were optimised for **ROC-AUC** via `GridSearchCV` with 5-fold `StratifiedKFold`, and evaluated **once** on the isolated hold-out test set (~67% / 33% split).

The three **boosting** models (CatBoost, LightGBM, XGBoost) clearly **outperform** every other approach and cluster tightly together (**0.725–0.726**).

### 8.2 Final model

The best estimator is **CatBoostClassifier** with `{depth: 6, iterations: 400, learning_rate: 0.2, auto_class_weights: None}`, reaching **ROC-AUC = 0.726** on the unseen test set. Notably, the top models did *not* benefit from class re-weighting (the best CatBoost and Decision Tree configurations both used no class weights), confirming that the mild imbalance was adequately addressed by stratification alone.

The absolute ROC-AUC (~0.73) is **modest but well above random (0.50)** — a direct and expected consequence of the weak per-feature correlations observed in the EDA. The macro-recall (~0.63), lower than the overall accuracy (~0.68), shows the minority "no-diabetes" class remains the harder one to capture.

### 8.3 Other tests

Beyond the final pipeline presented above, a few alternative preprocessing strategies were tested to validate the scaling choice. In particular, the whole notebook was re-run **without any feature scaling** and using a `RobustScaler` (which centres on the median and scales by the IQR, making it less sensitive to the outliers detected in Section 3.5) in place of the `StandardScaler`.

Comparing the resulting ROC-AUC scores, the `StandardScaler` **consistently delivered the best results** and was therefore kept in the final pipeline. The differences were small or non-existent the boosting models and tree-based models.

## 9. References

---

- **Diabetes Prediction Challenge:** <https://www.kaggle.com/competitions/playground-series-s5e12/data>
- **Original Diabetes Health Indicators Dataset:** <https://www.kaggle.com/datasets/mohankrishnathalla/diabetes-health-indicators-dataset/data>
- **Hyperparameter LightGBM:** <https://www.kaggle.com/code/mustafagerme/hyperparameter-tuning-with-gridsearchcv-for-lgbm>
- **Hyperparameter XGBoost:** <https://www.kaggle.com/code/tilii7/hyperparameter-grid-search-with-xgboost>
- **LightGBM:** <https://www.kaggle.com/code/prashant111/lightgbm-classifier-in-python>

---

Exported with [runcell](#) — convert notebooks to HTML or PDF anytime at [runcell.dev](https://runcell.dev).